Capestone Project – Approach Sheet

Task 1: Web Scraping

I have scraped 2 data set of jobs data from Naukri.com

I have scraped short 50 page data at a time to avoid error, I scrapped 2 times from same website with same code only had separate dataframe then I will append the data using Excel/Power Bi.

Steps followed :

1. Import essential libraries.

```
import numpy as np
import pandas as pd
import matplotlib as plt
import seaborn as sns
```

```
! pip install selenium
! pip install beautifulsoup4
```

```
Requirement already satisfied: selenium in c:\users\alpan\anaconda3\lib\site-packages (4.43.0)
Requirement already satisfied: certifi>=2026.1.4 in c:\users\alpan\anaconda3\lib\site-packages (from selenium) (2026.2.25)
Requirement already satisfied: trio<1.0,>=0.31.0 in c:\users\alpan\anaconda3\lib\site-packages (from selenium) (0.33.0)
Requirement already satisfied: trio-websocket<1.0,>=0.12.2 in c:\users\alpan\anaconda3\lib\site-packages (from selenium) (0.12.2)
Requirement already satisfied: typing_extensions<5.0,>=4.15.0 in c:\users\alpan\anaconda3\lib\site-packages (from selenium) (4.15.0)
Requirement already satisfied: urllib3<3.0,>=2.6.3 in c:\users\alpan\anaconda3\lib\site-packages (from urllib3[socks]<3.0,>=2.6.3->selenium) (2.6.3)
Requirement already satisfied: websocket-client<2.0,>=1.8.0 in c:\users\alpan\anaconda3\lib\site-packages (from selenium) (1.8.0)
Requirement already satisfied: attrs>=23.2.0 in c:\users\alpan\anaconda3\lib\site-packages (from trio<1.0,>=0.31.0->selenium) (24.3.0)
Requirement already satisfied: sortedcontainers in c:\users\alpan\anaconda3\lib\site-packages (from trio<1.0,>=0.31.0->selenium) (2.4.0)
Requirement already satisfied: idna in c:\users\alpan\anaconda3\lib\site-packages (from trio<1.0,>=0.31.0->selenium) (3.7)
Requirement already satisfied: outcome in c:\users\alpan\anaconda3\lib\site-packages (from trio<1.0,>=0.31.0->selenium) (1.3.0.post0)
Requirement already satisfied: sniffio>=1.3.0 in c:\users\alpan\anaconda3\lib\site-packages (from trio<1.0,>=0.31.0->selenium) (1.3.0)
Requirement already satisfied: cffi>=1.14 in c:\users\alpan\anaconda3\lib\site-packages (from trio<1.0,>=0.31.0->selenium) (1.17.1)
Requirement already satisfied: wsproto>=0.14 in c:\users\alpan\anaconda3\lib\site-packages (from trio-websocket<1.0,>=0.12.2->selenium) (1.3.2)
Requirement already satisfied: pysocks!=1.5.7,<2.0,>=1.5.6 in c:\users\alpan\anaconda3\lib\site-packages (from urllib3[socks]<3.0,>=2.6.3->selenium) (1.
7.1)
Requirement already satisfied: pycparser in c:\users\alpan\anaconda3\lib\site-packages (from cffi>=1.14->trio<1.0,>=0.31.0->selenium) (2.21)
Requirement already satisfied: h11<1,>=0.16.0 in c:\users\alpan\anaconda3\lib\site-packages (from wsproto>=0.14->trio-websocket<1.0,>=0.12.2->selenium)
(0.16.0)
Requirement already satisfied: beautifulsoup4 in c:\users\alpan\anaconda3\lib\site-packages (4.12.3)
Requirement already satisfied: soupsieve>1.2 in c:\users\alpan\anaconda3\lib\site-packages (from beautifulsoup4) (2.5)
```

2. Data collection: taking an empty list all data[] and storing the records into it from url of Naukri.com.

```
import bs4
from bs4 import BeautifulSoup
import requests
from selenium import webdriver
import time
```

```
driver = webdriver.Edge()
all_data1 = []
for page in range(1, 51):
    if page == 1:
        url = "https://www.naukri.com/data-science-jobs?src=gnbjobs_homepage_srch"
    else:
        url = f"https://www.naukri.com/data-science-jobs-{page}?src=gnbjobs_homepage_srch"
    driver.get(url)

    time.sleep(5)
    soup = BeautifulSoup(driver.page_source, "html.parser")
    jobs = soup.find_all("div", class_="srp-jobtuple-wrapper")

    print(f"Page {page}, Jobs found:", len(jobs))
    for job in jobs:
        try:
            Titles = job.find("a", class_="title").text.strip()
        except:
            Titles = ""
        try:
            Experience = job.find("span", class_="expwdth").text.strip()
        except:
            Experience = ""
        try:
            Location= job.find("span", class_="locWdth").text.strip()
        except:
            Location = ""
```

```
        Experience =
    try:
        Location= job.find("span", class_="locWdth").text.strip()
    except:
        Location = ""
    try:
        Company_name= job.find("a", class_="comp-name mw-25").text.strip()
    except:
        Company_name = ""
    try:
        Job_Educational_requirements= job.find("span", class_="job-desc ni-job-tuple-icon ni-job-tuple-icon-srp-description").text.strip()
    except:
        Job_Educational_requirements = ""
    try:
        Skills_required = job.find("ul", class_="tags-gt").text.strip()
    except:
        Skills_required= ""
    try:
        Total_reviews = job.find("a", class_="review ver-line").text.strip()
    except:
        Total_reviews= ""
    try:
        Rating = job.find("span", class_="main-2").text.strip()
    except:
        Rating= ""
    try:
        job_post_day = job.find("span", class_="job-post-day").text.strip()
    except:
        job_post_day=""
    all_data1.append([Titles, Experience, Location, Company_name, Job_Educational_requirements,Skills_required, Total_reviews, Rating, job_post_day])
```

```
Page 1, Jobs found: 20
Page 2, Jobs found: 20
Page 3, Jobs found: 20
Page 4, Jobs found: 20
```

Thus, I get records of 50 pages in one go.

3. Then I have created a dataframe df1 and stored the data in it and saved it as a csv file in my operating system.

```
df1 = pd.DataFrame(all_data1, columns=["Titles", "Experience", "Location", "Company_name", "Job_Educational_requirements",
"Skills_required", "Total_reviews", "Rating","job_post_day"])
print(len(all_data1))
df1.to_csv("naukri_jobslist1.csv", index=False)
print(f"Saved till page {page}")
```

```
1000
Saved till page 50
```

```
import os
print(os.getcwd())
```

```
C:\Users\alpan
```

```
df1.shape
```

```
(1000, 9)
```

```
print(len(Titles))
print(len(Experience))
print(len(Location))
print(len(Company_name))
print(len(Job_Educational_requirements))
print(len(Skills_required))
print(len(Total_reviews))
print(len(Rating))
print(len(job_post_day))
```

```
22
9
4
6
90
111
13
3
10
```

4. Then importing os I look for the location of the file in my operating system.
5. Using same method I have scraped data of 100 pages and got 1000,998 records in loop respectively that is 1998 in total.

```
Page 98, Jobs found: 20
Page 99, Jobs found: 20
Page 100, Jobs found: 20
```

```python
df2 = pd.DataFrame(all_data2, columns=["Titles", "Experience", "Location", "Company_name", "Job_Educational_requirements",
"Skills_required", "Total_reviews", "Rating","job_post_day"])
print(len(all_data2))
df2.to_csv("naukri_jobslist2.csv", index=False)
print(f"Saved till page {page}")
```

```
998
Saved till page 100
```

```python
import os
print(os.getcwd())
```

```
C:\Users\alpan
```

```python
import numpy as np
import pandas as pd
import matplotlib as plt
import seaborn as sns
import glob

files = [
    r"C:\Users\alpan\naukri_jobslist1.csv",
    r"C:\Users\alpan\naukri_jobslist2.csv"
]
df = pd.concat([pd.read_csv(f) for f in files], ignore_index=True)

df.to_csv("naukri_all_jobs.csv", index=False)
```

```python
df.shape
```

```
(1998, 9)
```

```python
import os
print(os.getcwd())
```

```
C:\Users\alpan
```

```python
df.head()
```

	Titles	Experience	Location	Company_name	Job_Educational_requirements	Skills_required	Total_reviews	Rating	job_post_day
0	Accenture is hiring For Data Science Specialist	6-10 Yrs	Bengaluru, Mumbai (All Areas)	Accenture	In this role you are required to do analysis a...	Data ScienceMachine LearningPythonArtificial I...	73086 Reviews	3.7	2 weeks ago
1	SENIOR MANAGER, DATA SCIENCE	12-17 Yrs	Bengaluru	Walmart	The Catalog Data Science Team at Walmart Globa...	Computer visionProduct engineeringManager Qual...	3118 Reviews	3.5	1 day ago
2	Advanced Data Science Consultant	6-9 Yrs	Hybrid - Pune, Gurugram, Bengaluru	ZS	Guide and manage project teams of Associates /...	data scienceData EngineeringEngineering ToolsR...	2727 Reviews	3.1	3+ weeks ago
3	Gen AI + Data Science Developer/Lead	8-13 Yrs	Hybrid - Noida, Pune	Ltimindtree	Experience Required . 3 to 15 years . Notice P...	Artificial IntelligenceStatistical ModelingMac...	26353 Reviews	3.6	1 week ago
4	Data Science Manager, gTech Ads Solutions	9-14 Yrs	Hyderabad, Bengaluru	Google	Masters degree in a quantitative discipline su...	Product engineeringdata scienceCodingAnalytica...	1891 Reviews	4.4	3+ weeks ago

```python
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1998 entries, 0 to 1997
Data columns (total 9 columns):
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1998 entries, 0 to 1997
Data columns (total 9 columns):
 #   Column                       Non-Null Count  Dtype
---  ------                       --------------  -----
 0   Titles                       1998 non-null   object
 1   Experience                   1959 non-null   object
 2   Location                     1994 non-null   object
 3   Company_name                 1677 non-null   object
 4   Job_Educational_requirements 1964 non-null   object
 5   Skills_required              1964 non-null   object
 6   Total_reviews                1677 non-null   object
 7   Rating                       1677 non-null   float64
 8   job_post_day                 1998 non-null   object
dtypes: float64(1), object(8)
memory usage: 140.6+ KB
```

6. Then proceeded with appending all the files into one file.

Appended the files using Power Query in MS Excel and splitted columns (experience), did some cleaning and removed Job_Educational_requirements column. as it is giving misleading values.

```
import numpy as np
import pandas as pd
import matplotlib as plt
import seaborn as sns
```

```
df= pd.read_csv(r"C:\Users\alpan\anaconda_projects\d066b04f-01a6-47f1-bb97-d2502c45bac7\naukri_all_jobslist.csv",encoding="latin-1")
df.head()
```

	Titles	Min.Experience	Max.Experience	Location	Company_name	Skills_required	Total_reviews.1	Rating	job_post_day
0	Advanced Data Science Consultant	6.0	9.0	Hybrid - Pune	ZS	data scienceData EngineeringEngineering ToolsR...	2725.0	3.1	3 weeks ago
1	Advanced Data Science Consultant	6.0	9.0	Gurugram	ZS	data scienceData EngineeringEngineering ToolsR...	2725.0	3.1	3 weeks ago
2	Advanced Data Science Consultant	6.0	9.0	Bengaluru	ZS	data scienceData EngineeringEngineering ToolsR...	2725.0	3.1	3 weeks ago
3	Urgent: Job Opportunity with PWC-AC-Data scien...	4.0	8.0	Hybrid - Kolkata	PwC Service Delivery Center	Data SciencePythonAI MLCloudMachine LearningAr...	12482.0	3.3	1 day ago
4	Urgent: Job Opportunity with PWC-AC-Data scien...	4.0	8.0	Hyderabad	PwC Service Delivery Center	Data SciencePythonAI MLCloudMachine LearningAr...	12482.0	3.3	1 day ago

```
df.columns
```

```
Index(['Titles', 'Min.Experience', 'Max.Experience', 'Location',
       'Company_name', 'Skills_required', 'Total_reviews.1', 'Rating',
       'job_post_day'],
      dtype='object')
```

7. In Excel using power query editor I have splitted location column values into rows to have consistent values In the dataset. Thus, total records increased from 1998 to 2417.

```
df.shape
```

```
(2417, 9)
```

8. Then I used Power query editor of MS Excel to split columns and have consistent non duplicate columns, also I have splitted location column values into rows for better understanding of the data.

```
df.columns
```

```
Index(['Titles', 'Min.Experience', 'Max.Experience', 'Location',
       'Company_name', 'Job_Educational_requirements', 'Skills_required',
       'Total_reviews.1', 'Rating', 'job_post_day'],
      dtype='object')
```

```
df.shape
```

```
(5894, 10)
```

- **Data cleaning and pre-processing**
    1. **Handling missing values**

Data Cleaning and preprocessing

1. Handling missing values

```
df.isnull().sum()
```

```
Titles               0
Min.Experience      76
Max.Experience      76
Location             6
Company_name       557
Skills_required     71
Total_reviews.1    557
Rating             557
job_post_day         0
dtype: int64
```

```
df = df.dropna(subset=["Min.Experience", "Max.Experience","Location","Company_name", "Skills_required",
                       "Total_reviews.1","Rating"])
```

```
df.isnull().sum()
```

```
Titles             0
Min.Experience     0
Max.Experience     0
Location           0
Company_name       0
Skills_required    0
Total_reviews.1    0
Rating             0
job_post_day       0
dtype: int64
```

```
df.shape
```

```
(1802, 9)
```

Now I have no null values in the data. I removed them because it is a real-world project, and mentioning anything hypothetical would lead to discrepancies.

2. **Handling duplicate records:** removed duplicate values to have unique values in the data set for correct analysis.

2. Handling duplicate records

```
df.columns
```

```
Index(['Titles', 'Min.Experience', 'Max.Experience', 'Location',
       'Company_name', 'Skills_required', 'Total_reviews.1', 'Rating',
       'job_post_day'],
      dtype='object')
```

```
df.duplicated(subset=['Titles', 'Min.Experience', 'Max.Experience', 'Location',
       'Company_name', 'Skills_required','Total_reviews.1', 'Rating', 'job_post_day']).sum()
```

```
np.int64(7)
```

```
df.drop_duplicates(subset=['Titles', 'Min.Experience', 'Max.Experience', 'Location',
       'Company_name', 'Skills_required',
       'Total_reviews.1', 'Rating', 'job_post_day'], inplace=True)
df.shape
```

```
(1795, 9)
```

3. **Inconsistent Formats**: Many columns have inconsistent formats it is formatted using different python functions.

- **Skills**

Skills required

In Skills required the values are messy it needs to be cleaned or to be in well format to be understood easily.

```
df['Skills_required'].value_counts()
```

```
Skills_required
metadatapythondata analysisdata managementdata analyticsdata validationconfigurationdata mining
25
algorithmspythonmodelingdata analysisdata analyticsnatural language processingpredictivemachine learning
24
master datametadatapythondata analysisdata managementdata analyticsdata validationconfiguration
14
Trainingdeep learningTeam managementdata scienceGCPMachine learningDeploymentMonitoring
10
Machine LearningPythonMatplotlibJupyter NotebookDeep LearningNumpyScikit-LearnSQL
9

..
SQL queriesAutomationData managementFinanceIssue resolutionData qualityData analyticsData Analyst
1
Data analysisSASReport writingData collectionPackagingpower biAdvanced ExcelData analytics
1
Analytical skillsData modelingData analyticsData AnalystAnalyst 2StatisticsAnalytical SkillAnalytics
1
AutomationData analysisChange managementOperational excellenceAnalyticalData qualityOperationsFinancial services
1
Data ScienceTensorflowArtificial IntelligenceMachine Learning AlgorithmsMachine LearningAlgorithm DevelopmentNatural Language ProcessingComputer Vision
1
Name: count, Length: 1297, dtype: int64
```

As the data is huge i will create a master of skill sets and then use skill dictionary to search for the skill in the skill required column.

```
skills_master = [
    "python","sql","machine learning","deep learning",
    "natural language processing","data science","data engineering",
    "data analysis","pandas","numpy","tensorflow","pytorch",
    "azure","aws", "computer vision", "tableau","power bi","excel","statistics","Predictive modeling",
    "time series","git", "kubernetes","docker","llm","generative ai","Data modelling","rca", "ai solutions",
    "ai cloud platforms", "artificial intelligence",
    "Data visualisation", "Data processing","python libraries","Statistical analysis"
]
```

Using re library i clean the text and convert it into lower case .

```
import re

def clean_text(text):
    text = str(text).lower()
    text = re.sub(r'[^a-zA-Z ]', ' ', text)
    return text
```

then create a function extract-skills and if the same skill is found on text(skill required column) it will be stored in found list and then match it with the clean column where cleaned records are stoered.

```
def extract_skills(text):
    found = []
    for skill in skills_master:
        if skill in text:
            found.append(skill)
    return found

df['clean'] = df['Skills_required'].apply(clean_text)
df['skills_extracted'] = df['clean'].apply(extract_skills)
```

Once the skills are matching with the skill reuired (clean) column and the extracted skills we'll count the skills

```
from collections import Counter

all_skills = []
for row in df['skills_extracted']:
    all_skills.extend(row)

top_skills = Counter(all_skills)
top_skills_df = pd.DataFrame(top_skills.items(), columns=['Skill','Count']) \
                    .sort_values(by='Count', ascending=False)

top_skills_df.head(10)
```

	Skill	Count
3	machine learning	840
2	python	821
0	data science	651
5	data analysis	548
6	sql	360
13	natural language processing	295
4	artificial intelligence	233
7	power bi	231
9	deep learning	169
8	azure	85

"I flattened the extracted skill lists and used a frequency count to identify the most in-demand skills across job postings."

Now the skill column is cleaned and each skill is separated by comma.

```python
df.head()
```

Titles	Min.Experience	Max.Experience	Location	Company_name	Skills_required	Total_reviews.1	Rating	job_post_day	clean	skills_extracted
ranced Data cience sultant	6.0	9.0	Hybrid - Pune	ZS	data scienceData EngineeringEngineering ToolsR...	2725.0	3.1	3 weeks ago	data sciencedata engineeringengineering toolsr...	[data science, data engineering]
ranced Data cience sultant	6.0	9.0	Gurugram	ZS	data scienceData EngineeringEngineering ToolsR...	2725.0	3.1	3 weeks ago	data sciencedata engineeringengineering toolsr...	[data science, data engineering]
ranced Data cience sultant	6.0	9.0	Bengaluru	ZS	data scienceData EngineeringEngineering ToolsR...	2725.0	3.1	3 weeks ago	data sciencedata engineeringengineering toolsr...	[data science, data engineering]
nt: Job rtunity PWC- -Data scien...	4.0	8.0	Hybrid - Kolkata	PwC Service Delivery Center	Data SciencePythonAI MLCloudMachine LearningAr...	12482.0	3.3	1 day ago	data sciencepythonai mlcloudmachine learningar...	[python, machine learning, data science, artif...
nt: Job rtunity PWC- -Data scien...	4.0	8.0	Hyderabad	PwC Service Delivery Center	Data SciencePythonAI MLCloudMachine LearningAr...	12482.0	3.3	1 day ago	data sciencepythonai mlcloudmachine learningar...	[python, machine learning, data science, artif...

```python
df_exploded = df.explode('skills_extracted')
df_exploded['skills_extracted'] = df_exploded['skills_extracted'].str.lower().str.strip()
```

```python
skill_counts = df_exploded['skills_extracted'].value_counts()
```

```python
df['skills_extracted'].value_counts()
```

```
skills_extracted
[]                                                                  101
[data analysis]                                                      80
[machine learning, data science]                                     69
[data science]                                                       64
[python, data analysis]                                              63
                                                                   ...
[python, sql, data science, excel]                                    1
[data science, data analysis, power bi, git]                          1
[python, machine learning, natural language processing, kubernetes]   1
[machine learning, data science, computer vision, artificial intelligence]   1
[python, machine learning, data science, data analysis, numpy]        1
Name: count, Length: 501, dtype: int64
```

Blank space [ ] is there in the values so these blank values is replaced by vdata science as it is the most required skill in data science job.

```python
df['skills_extracted'] = df['skills_extracted'].apply(lambda x: x if len(x) > 2 else ['data science'])
df['skills_extracted'].value_counts()
```

```
skills_extracted
[data science]                                                                          805
[python, machine learning, natural language processing, data analysis]                   32
[python, machine learning, data science]                                                 26
[machine learning, deep learning, data science]                                          22
[python, deep learning, natural language processing, data science, computer vision]      21
                                                                                       ...
[python, sql, machine learning, natural language processing, data engineering, artificial intelligence]   1
[machine learning, data analysis, computer vision]                                       1
[sql, deep learning, numpy, tensorflow]                                                  1
[sql, machine learning, tableau, power bi]                                               1
[data engineering, data analysis, power bi]                                              1
```

```
[data engineering, data analysis, power bi]                                    1
Name: count, Length: 417, dtype: int64
```

```
df_exploded['skills_extracted'].value_counts()
```

```
skills_extracted
machine learning               840
python                         821
data science                   651
data analysis                  548
sql                            360
natural language processing    295
artificial intelligence        233
power bi                       231
deep learning                  169
azure                           85
git                             85
generative ai                   73
data engineering                71
numpy                           69
computer vision                 62
tableau                         62
tensorflow                      59
excel                           56
statistics                      54
aws                             47
kubernetes                      39
ai solutions                    36
llm                             34
time series                     32
pytorch                         26
docker                          10
rca                              5
pandas                           4
Name: count, dtype: int64
```

```
df.shape
```

```
(1795, 11)
```

Now the skill column is cleaned and well formatted.

- **Location**

Location

```
df['Location'].value_counts()
```

```
Location
Bengaluru                        505
Hyderabad                        153
Gurugram                         150
 Bengaluru                       108
Chennai                          104
                                ...
Thane                              1
 Lucknow                           1
Pune(Viman Nagar)                  1
Bengaluru(Sector 7 HSR Layout)     1
 Kursela                           1
Name: count, Length: 130, dtype: int64
```

Location names are not is correct format, this column also requires processing.

```
df['Location'] = df['Location'].str.lower().str.strip()
```

```
location_map = {
    'new delhi': 'delhi',
    'delhi ncr': 'delhi',
    'delhi / ncr':'noida',
    'bangalore': 'bengaluru',
    'navi mumbai': 'mumbai',
    'india':'remote',
    'hybrid - hyderabad':'hyderabad',
    'hybrid - bengaluru':'bengaluru',
    'hybrid - pune':'pune',
    'hybrid - gurugram':'gurugram',
    'hybrid - chennai':'chennai',
    'hybrid - kolkata':'kolkata',
    'mumbai (all areas)': 'mumbai',
    'hybrid - bengaluru(koramangala)':'bengaluru',
    'hybrid - mumbai (all areas)':'mumbai'

}

df['Location'] = df['Location'].str.lower().replace(location_map)
```

```
df['Location'].value_counts()
```

```
Location
bengaluru                       634
hyderabad                       202
gurugram                        182
pune                            155
chennai                         152
                               ...
nashik                            1
thane                             1
pune(viman nagar)                 1
bengaluru(sector 7 hsr layout)    1
kursela                           1
```

After the Formatting of the inconsistent columns we move to the feature engineering task.

4. Feature Engineering:

- **Experience level**

Segmented jobs based on minimum experience into Entry level, mid level and senior level.

```
df.describe()
```

	Min.Experience	Max.Experience	Total_reviews.1	Rating
count	1795.000000	1795.000000	1795.000000	1795.000000
mean	4.532033	8.432869	14257.141504	3.598050
std	3.115118	3.879789	26956.294329	0.498215
min	0.000000	1.000000	2.000000	1.000000
25%	2.000000	6.000000	62.000000	3.400000
50%	4.000000	8.000000	704.000000	3.700000
75%	6.000000	10.000000	8566.000000	3.900000
max	20.000000	30.000000	113460.000000	5.000000

```python
def exp_level(row):
    if row['Min.Experience'] >=0  and row['Max.Experience'] <= 3:
        return 'Entry level'
    elif row['Min.Experience'] >= 4 and row['Max.Experience'] <= 10:
        return 'Mid level'
    else:
        return 'Senior level'

df['experience_level'] = df.apply(exp_level, axis=1)
df['experience_level']
```

```
0          Mid level
1          Mid level
2          Mid level
3          Mid level
4          Mid level
            ...
2412    Senior level
2413    Senior level
2414    Senior level
2415    Senior level
2416    Senior level
Name: experience_level, Length: 1795, dtype: object
```

```python
df.columns
```

```
Index(['Titles', 'Min.Experience', 'Max.Experience', 'Location',
       'Company_name', 'Skills_required', 'Total_reviews.1', 'Rating',
       'job_post_day', 'clean', 'skills_extracted', 'experience_level'],
      dtype='object')
```

```python
df['experience_level'].value_counts()
```

```
experience_level
Senior level    1009
Mid level        623
Entry level      163
Name: count, dtype: int64
```

- **Top Locations:**

Top 10 Locations

```python
top_locs = df['Location'].value_counts().head(10).index

df['location_group'] = df['Location'].apply(
    lambda x: x if x in top_locs else 'Others')
df['location_group'].value_counts()
```

```
location_group
bengaluru    634
hyderabad    202
Others       192
gurugram     182
pune         155
chennai      152
mumbai       115
delhi         55
remote        41
kolkata       36
noida         31
Name: count, dtype: int64
```

```python
df.columns
```

```
Index(['Titles', 'Min.Experience', 'Max.Experience', 'Location',
       'Company_name', 'Skills_required', 'Total_reviews.1', 'Rating',
       'job_post_day', 'clean', 'skills_extracted', 'experience_level',
       'location_group'],
      dtype='object')
```

➢ Review Segment

Total reviews from the jobs are segmente into 3 groups Low reviews, Medium reviews, High reviews

```
df.describe()
```

	Min.Experience	Max.Experience	Total_reviews.1	Rating
count	1795.000000	1795.000000	1795.000000	1795.000000
mean	4.532033	8.432869	14257.141504	3.598050
std	3.115118	3.879789	26956.294329	0.498215
min	0.000000	1.000000	2.000000	1.000000
25%	2.000000	6.000000	62.000000	3.400000
50%	4.000000	8.000000	704.000000	3.700000
75%	6.000000	10.000000	8566.000000	3.900000
max	20.000000	30.000000	113460.000000	5.000000

Rather than guessing the threshold, Here qcut used to segment the data (rows) into approx 33% in each segment.

```
df['review_segment'] = pd.qcut(
    df['Total_reviews.1'],
    q=3,
    labels=['Low', 'Medium', 'High'])
```

```
df['review_segment'].value_counts()
```

```
review_segment
Low       624
High      597
Medium    574
Name: count, dtype: int64
```

➢ Rating

```
df['Rating'] = pd.to_numeric(df['Rating'], errors='coerce')
```

```
df['rating_segment'] = pd.qcut(
    df['Rating'],
    q=3,
    labels=['Low', 'Medium', 'High'])
```

```
df['rating_segment'].value_counts()
```

```
rating_segment
Low       724
Medium    571
High      500
Name: count, dtype: int64
```

EDA And Visualization

Exploratory Data Analysis & Visualization

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 1795 entries, 0 to 2416
Data columns (total 15 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Titles            1795 non-null   object
 1   Min.Experience    1795 non-null   float64
 2   Max.Experience    1795 non-null   float64
 3   Location          1795 non-null   object
 4   Company_name      1795 non-null   object
 5   Skills_required   1795 non-null   object
 6   Total_reviews.1   1795 non-null   float64
 7   Rating            1795 non-null   float64
 8   job_post_day      1795 non-null   object
 9   clean             1795 non-null   object
 10  skills_extracted  1795 non-null   object
 11  experience_level  1795 non-null   object
 12  location_group    1795 non-null   object
 13  review_segment    1795 non-null   category
 14  rating_segment    1795 non-null   category
dtypes: category(2), float64(4), object(9)
memory usage: 200.1+ KB
```

```
df.shape
```

```
(1795, 15)
```

Statistical Analysis

```
df.describe()
```

	Min.Experience	Max.Experience	Total_reviews.1	Rating
count	1795.000000	1795.000000	1795.000000	1795.000000
mean	4.532033	8.432869	14257.141504	3.598050
std	3.115118	3.879789	26956.294329	0.498215
min	0.000000	1.000000	2.000000	1.000000
25%	2.000000	6.000000	62.000000	3.400000
50%	4.000000	8.000000	704.000000	3.700000
75%	6.000000	10.000000	8566.000000	3.900000
max	20.000000	30.000000	113460.000000	5.000000

Null Values

```
df.isnull().sum()
```

```
Titles              0
Min.Experience      0
Max.Experience      0
Location            0
Company_name        0
Skills_required     0
Total_reviews.1     0
Rating              0
job_post_day        0
clean               0
skills_extracted    0
experience_level    0
location_group      0
review_segment      0
rating_segment      0
dtype: int64
```

Renaming of Columnnames

```
df.columns
```

```
Index(['Titles', 'Min.Experience', 'Max.Experience', 'Location',
       'Company_name', 'Skills_required', 'Total_reviews.1', 'Rating',
       'job_post_day', 'clean', 'skills_extracted', 'experience_level',
       'location_group', 'review_segment', 'rating_segment'],
      dtype='object')
```

```
df.rename(columns= {'Titles':'Job_title', 'Min.Experience':'Min_Experience', 'Max.Experience':'Max_Experience',
       'Total_reviews.1': 'Total_reviews','skills_extracted':'skills'},inplace = True )
```

```
df.head()
```

	Job_title	Min_Experience	Max_Experience	Location	Company_name	Skills_required	Total_reviews	Rating	job_post_day	clean	skills
0	Advanced Data Science Consultant	6.0	9.0	pune	ZS	data scienceData EngineeringEngineering ToolsR...	2725.0	3.1	3 weeks ago	data sciencedata engineeringengineering toolsr...	[data science
	Advanced					data scienceData				data sciencedata	

Dropping unnecessary columns

```
df.drop(columns=['clean','Skills_required'],inplace=True)
```

```
df.columns
```

```
Index(['Job_title', 'Min_Experience', 'Max_Experience', 'Location',
       'Company_name', 'Total_reviews', 'Rating', 'job_post_day', 'skills',
       'experience_level', 'location_group', 'review_segment',
       'rating_segment'],
      dtype='object')
```

Univariate analysis

```
num_cols = df.select_dtypes(include=['int64', 'float64']).columns
cat_cols = df.select_dtypes(include=['object', 'category']).columns
```

```
num_cols
```

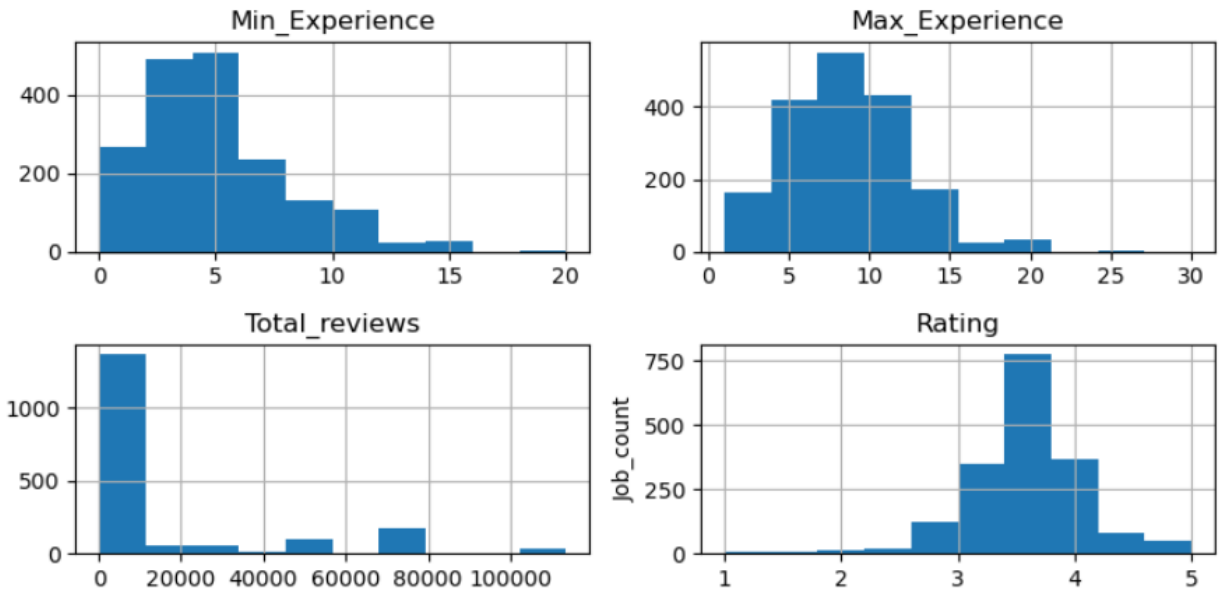
```
Index(['Min_Experience', 'Max_Experience', 'Total_reviews', 'Rating'], dtype='object')
```

```
cat_cols
```

```
Index(['Job_title', 'Location', 'Company_name', 'job_post_day', 'skills',
       'experience_level', 'location_group', 'review_segment',
       'rating_segment'],
      dtype='object')
```

Distribution of Data in numerical columns

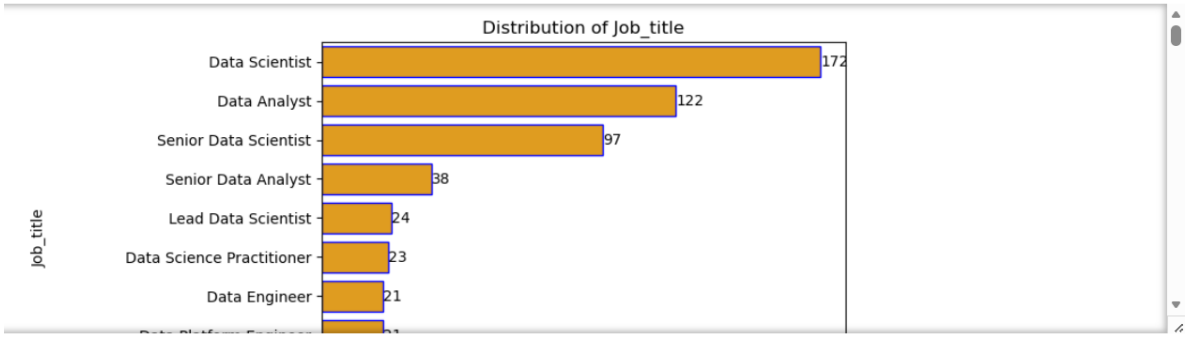
```
import matplotlib.pyplot as plt
import seaborn as sns

df[num_cols].hist(figsize=(8,4))
plt.ylabel("Job_count")
plt.tight_layout()
plt.show()
```



Then checked for the outliers in numerical columns using Boxplot.

Distribution of Data in Categorical columns

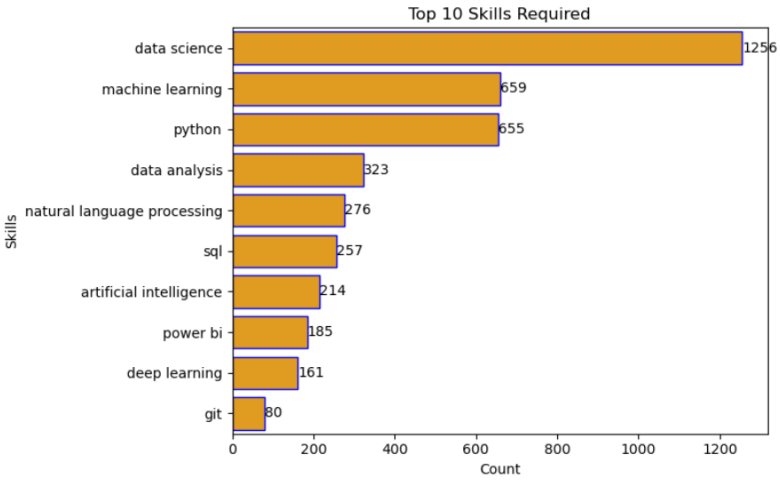
```
]: for col in cat_cols:
    cc=sns.countplot(y=col, data=df, order=df[col].value_counts().head(10).index,color = 'orange', edgecolor ='blue')
    cc.bar_label(cc.containers[0])
    plt.yticks(rotation=0)
    plt.title(f'Distribution of {col}')
    plt.show()
```



Using countplot visualised the data distribution in categorical columns.

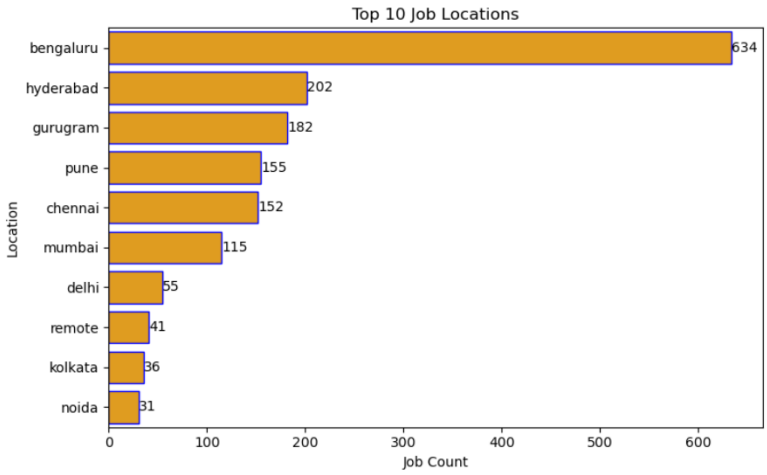
Then created some visualization:

1. **Top 10 Skills: Barplot**



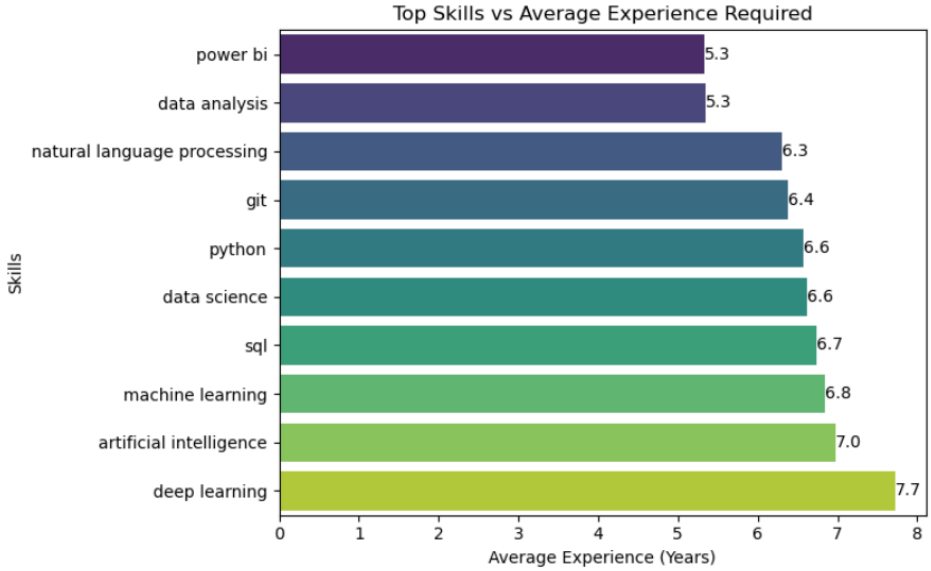
Data Science is the top skill required in the job market for Data Science and Analysis domain, followed by other combinations of skills.

2. **Total job counts per location: Barplot**



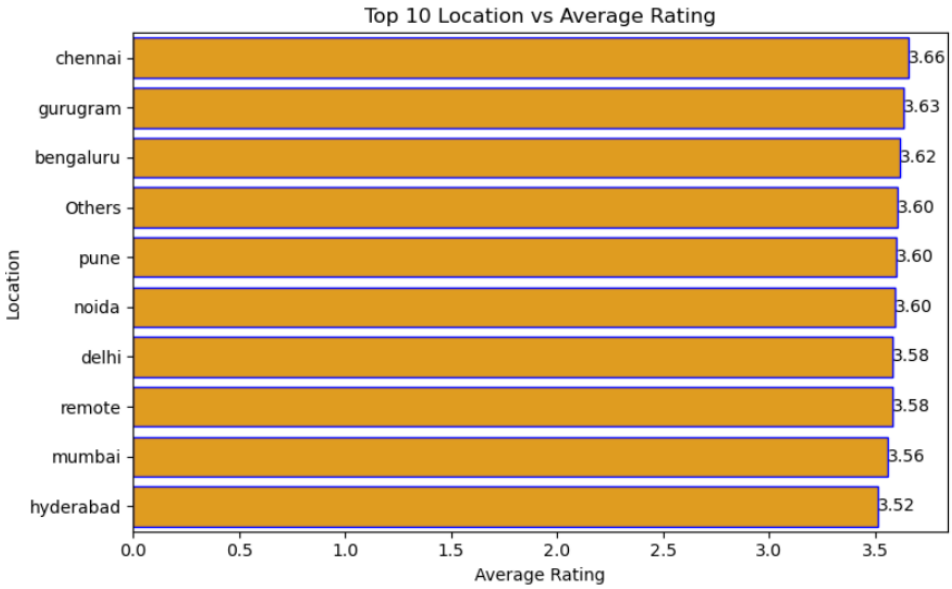
Bengaluru is the hub of Data Science jobs followed by Hyderabad, Gurugram, and Pune.

3. **Skills vs Experience: created column avg_experience and then using barplot visualized Top 10 skills VS Average experience required.**



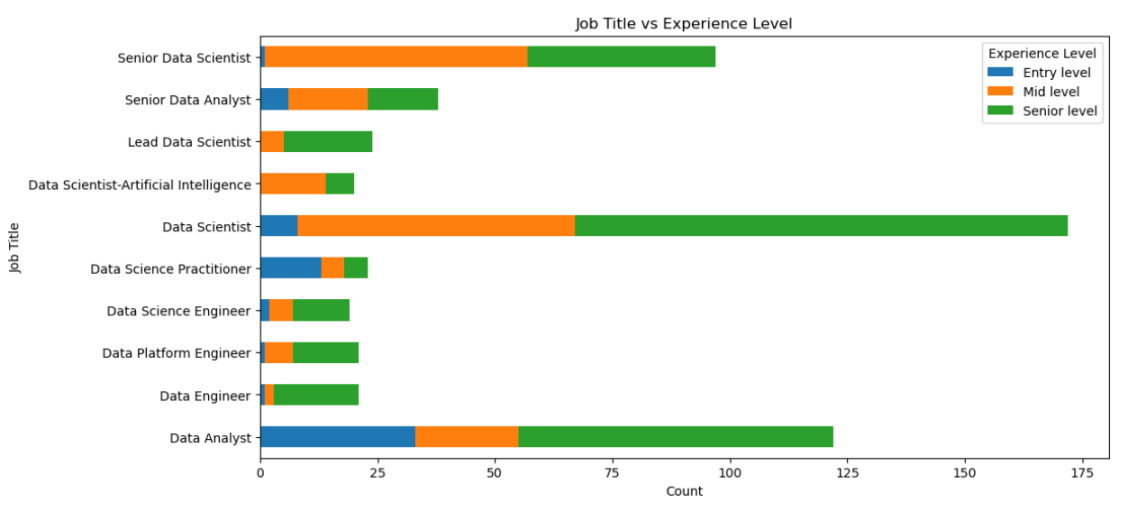
An increase in the number and complexity of required skills correlates with higher experience requirements, highlighting the importance of skill stacking for senior roles.

4. **Location vs Rating**

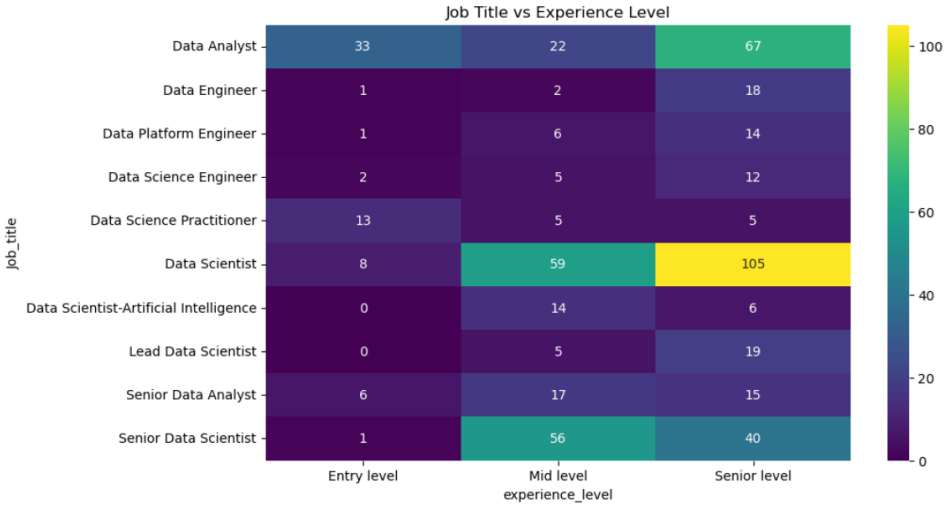


Location-based rating analysis suggests that while metro cities dominate hiring, smaller locations (others) may offer comparatively better work satisfaction, though this could be influenced by sample size bias.

5. **Job title VS Experience level: Horizontal barplot with experience level as hyperparameter (hue)**



Then using Heatmap found the correlation between job_title and experience level.



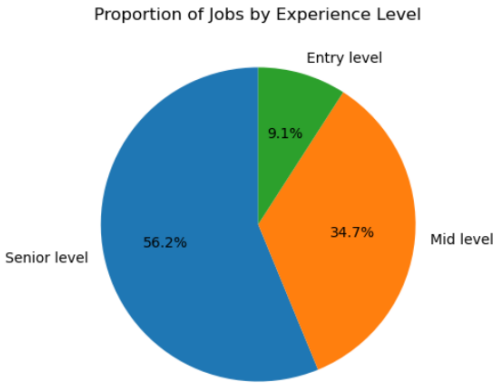
--> Most Data Analyst roles are concentrated in Entry and Mid-level positions.

--> Data Scientist roles skew toward Mid and Senior levels.

--> Senior roles are fewer but require specialized skillsets.

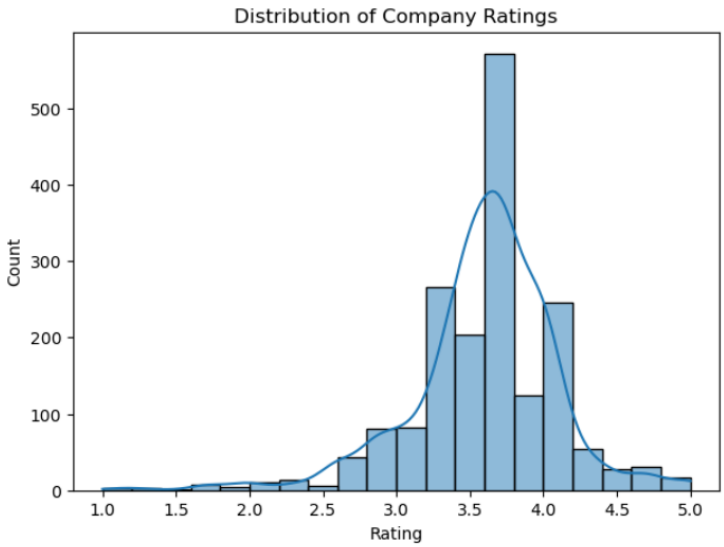
--> Entry-level opportunities are limited compared to mid-level demand.

6. Experience level: Pie Chart



--> Majority of job opportunities are concentrated in the senior-level experience (more than 10 years), indicating strong demand for highly experienced professionals.

7. Rating Distribution: Histplot

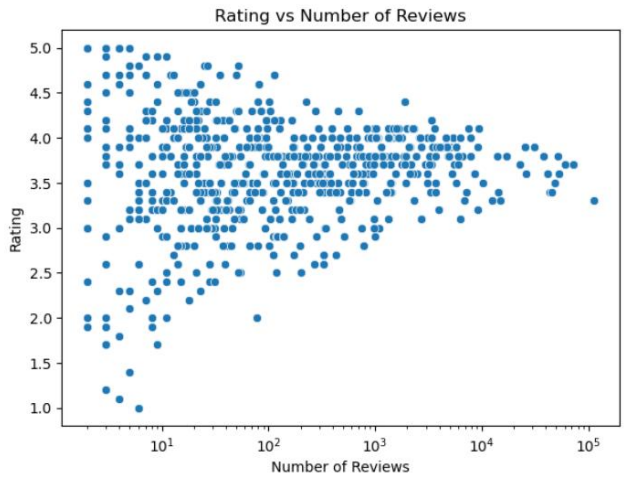


The job market is dominated by companies with average-to-good ratings, suggesting stable but competitive working environments rather than exceptional workplaces.

8. Review vs rating : Regression plot (regplot) to tell the relationship between the two.

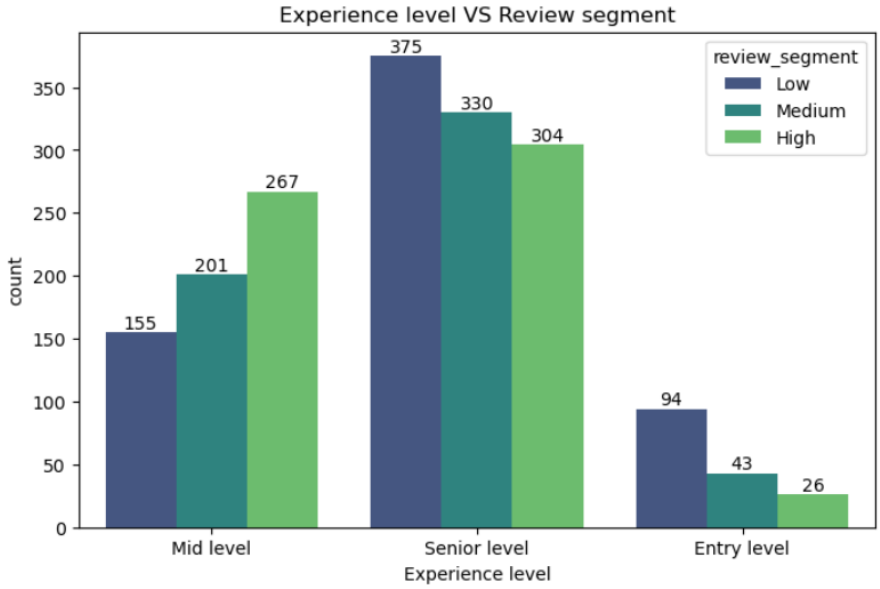
There is a very weak positive relationship between company reviews and ratings, indicating that higher review counts do not significantly influence company ratings.

Scatter plot for total reviews and rating



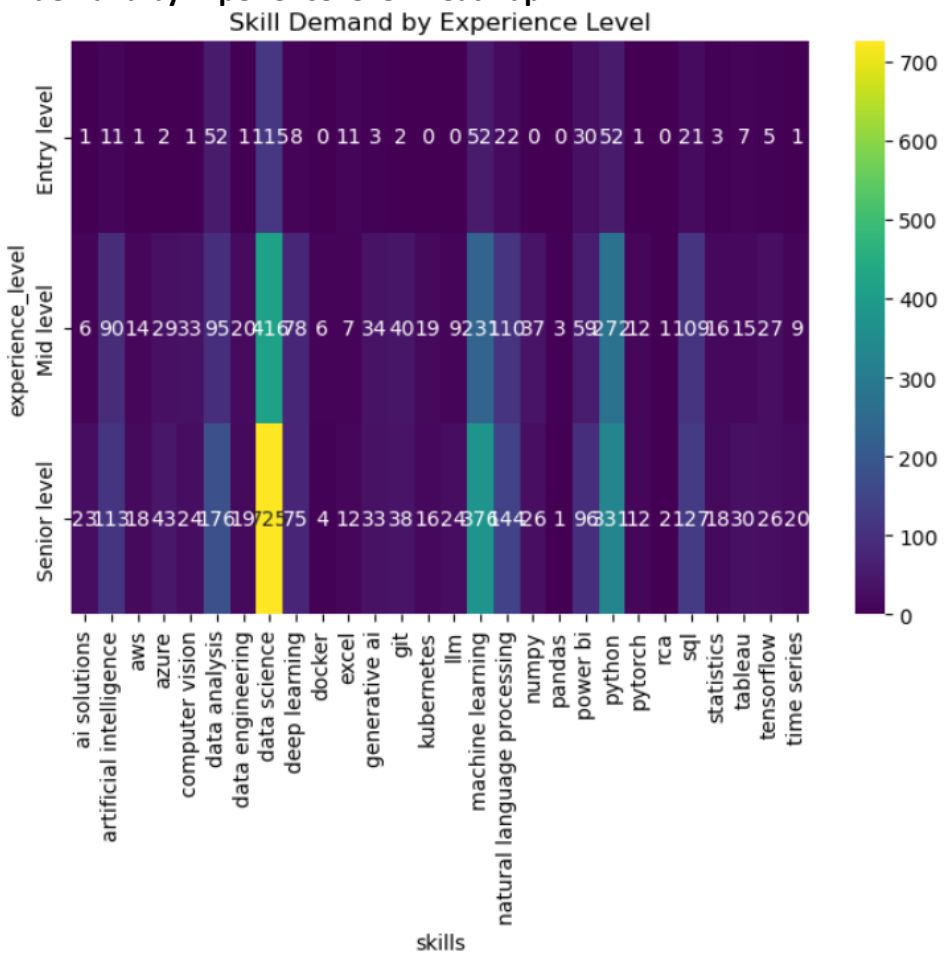
--> Companies with a higher number of reviews tend to have more stable and moderate ratings, indicating reduced bias compared to low-review companies

9. Experience level vs review segment: Countplot



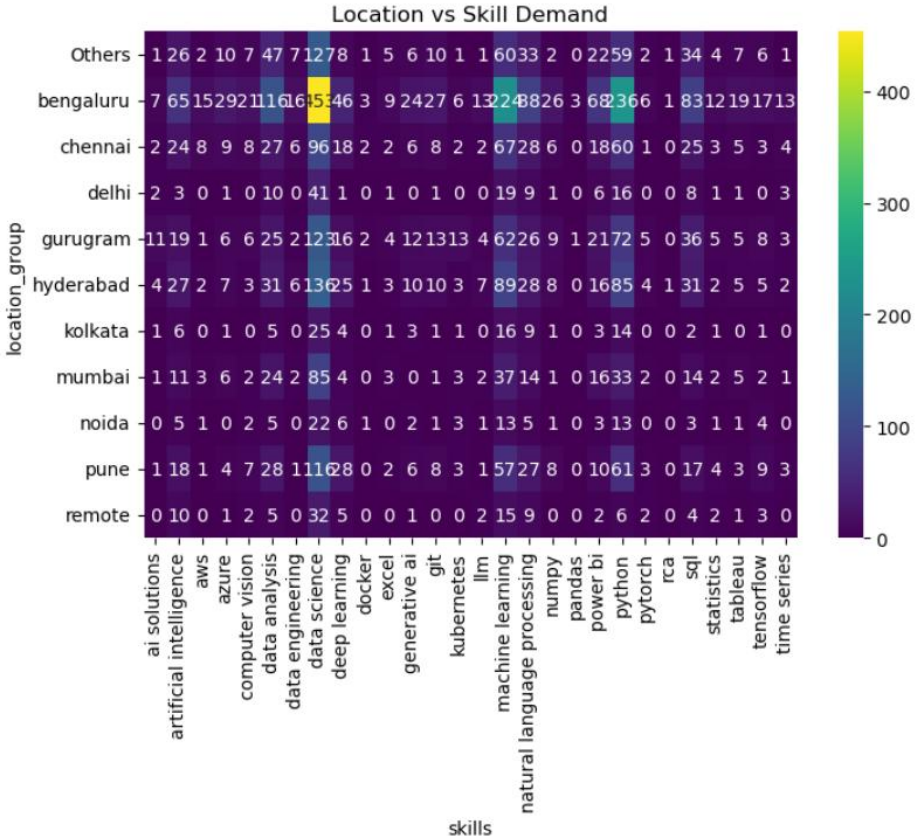
--> jobs in the high-review segment are more concentrated in mid and senior levels, suggesting that well-established companies prefer experienced candidates.

10. Skill demand by Experience level: Heatmap



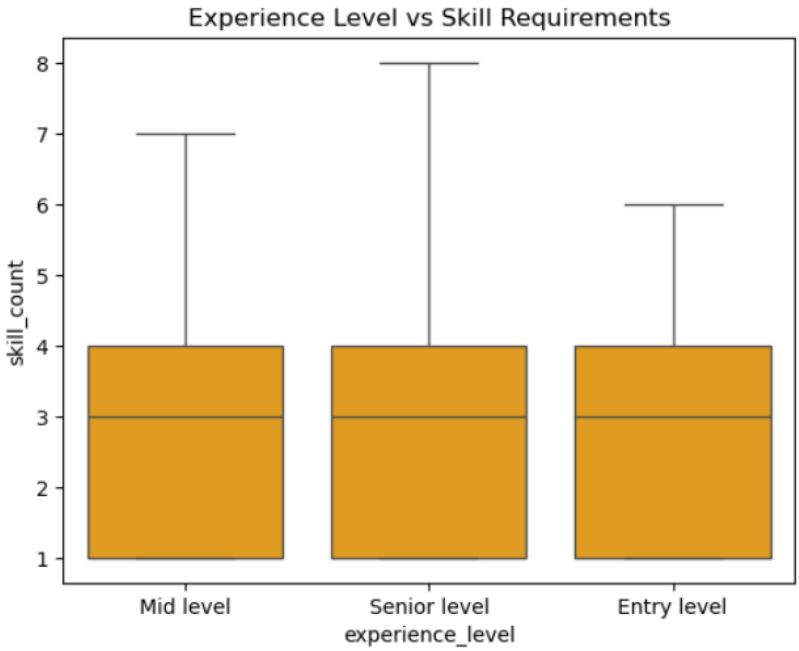
--> Senior-level roles focus on Data Science, Machine Learning, and Python, while entry-level roles demand Data Analysis, Data Science, Power BI, SQL, and Excel skills.

11. Skill demand by Location: Heatmap



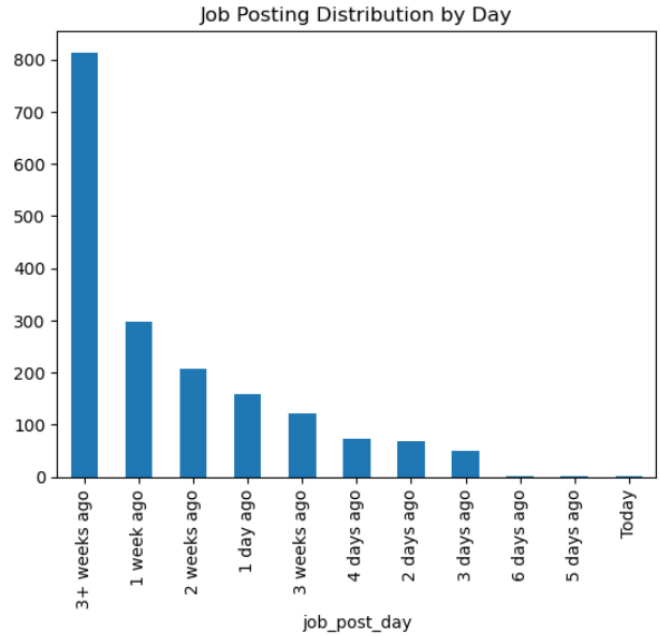
--> Tech hubs show higher demand for ML and Data Science

12. Experience level by skill requirement: Boxplot



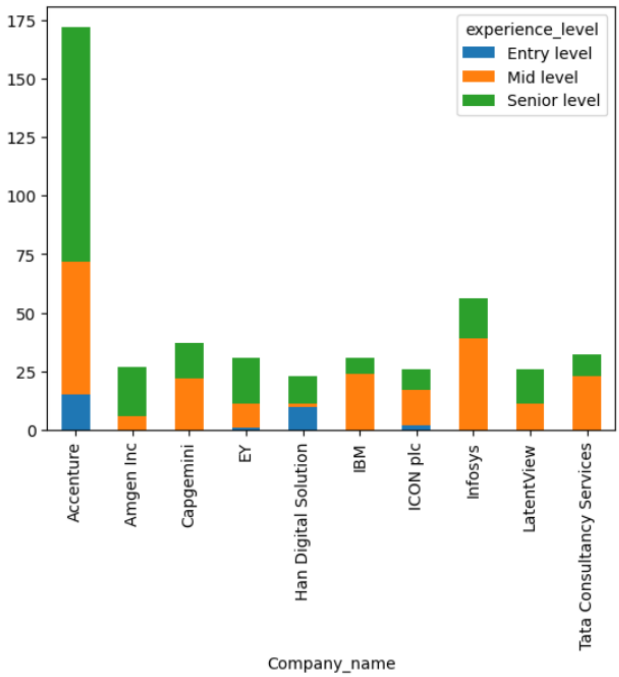
--> Higher experience roles require broader skillsets

13. Job Posting trends over time : Barplot



--> Most job [postings in the dataset are older than 3 weeks, with very few recent listings. this indicates that the data is heavily skewed toward outdated jobs and may not accurately reflect current job market activity.

14. Company hiring behavior: barplot

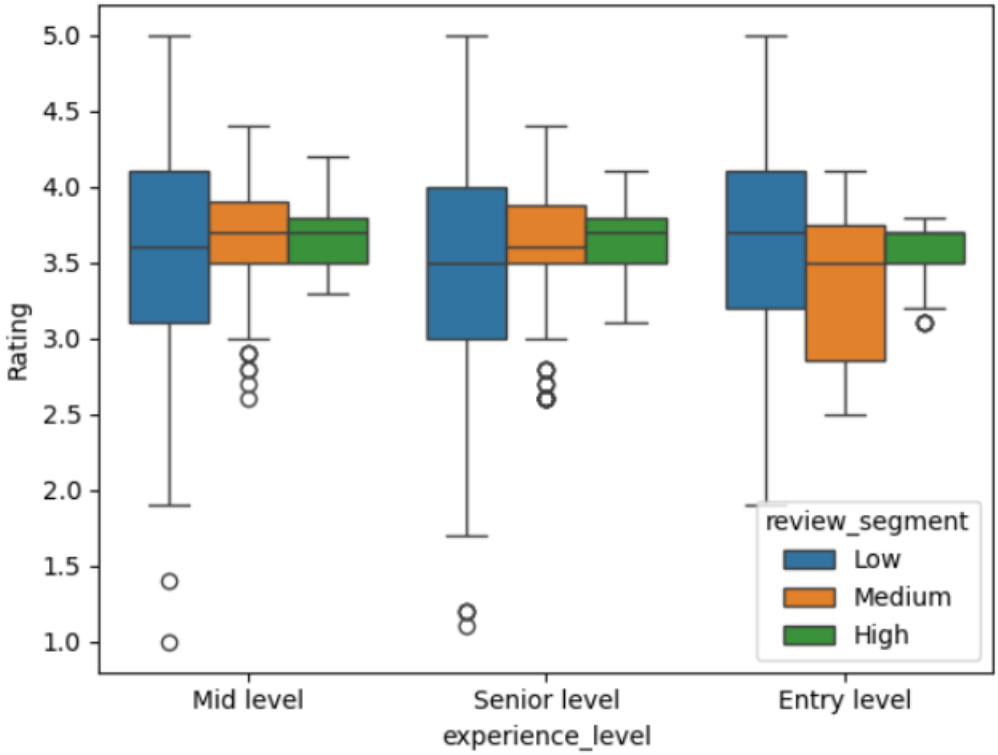


--> Most companies, especially Accenture have a higher number of openings for senior-level and mid-level roles compared to Entry-level positions, indicating a stronger demand fr experienced professionals in the job market.

15. Multidimensional Analysis: Boxplot

```
sns.boxplot(x='experience_level', y='Rating', hue='review_segment', data=df)
```

```
<Axes: xlabel='experience_level', ylabel='Rating'>
```



--> Highly reviewed companies maintain stable ratings across experience levels.

Model Building and Evaluation

As the data does not have any dependent column or any column that has binary data as such, A classification model was used to predict the experience level of a job based on skills, company information, and job attributes.

Experience level has 3 datavalues: Entry level, mid level and senior level .

Steps followed in model building :

1. **Feature selection:**
   df_model = df[['skills', 'location_group', 'Total_reviews', 'Rating', 'experience_level']]
   I have selected relevant columns to avoid noise and improve model performance.
2. **Converted skills which is in list to text format:**
   df_model['skills'] = df_model['skills'].apply(lambda x: ' '.join(x))
   because ML models require text-formatted data rather list.
3. **Label Encoding:**
   le = LabelEncoder()
   df_model['experience_level'] = le.fit_transform(df_model['experience_level'])
   Models need numbers, not text labels.
   Entry → 0
   Mid→ 1
   Senior→ 2
4. Splitting the data into train and test split: Train-Test split

```
from sklearn.model_selection import train_test_split
X = df_model.drop('experience_level', axis=1)
y = df_model['experience_level']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
```

Train on 75% of data and test on 25% unseen data.

5. Preprocessing the data

```
preprocessor = ColumnTransformer([
   ('text', TfidfVectorizer(), 'skills'),
   ('cat', OneHotEncoder(handle_unknown='ignore'), ['location_group']),
   ('num', 'passthrough', ['Total_reviews', 'Rating'])])
```

For skills : TFIDF that converts text→ numerical vectors and captures importance of words.

For Location: One hot encoding: converts categories → binary columns

And for rating and reviews they are passed as it is no changes done.

6. Model creation: pipeline is crested to combine Preprocessing and model training

```
\model = Pipeline([
('preprocessor', preprocessor),
('classifier', RandomForestClassifier())
])
```

It will clean the data make it scalable and avoid data leakage.

7. Model training: It will learn the pattern of the datset

```
model.fit(X_train, y_train)
```

8. Model Prediction: It will predict the experience level of unseen jobs.

```
y_pred = model.predict(X_test)
```

9. Model Evaluation:

```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

print("Accuracy:", accuracy_score(y_test, y_pred))

print(classification_report(y_test, y_pred))
```

Accuracy(Overall correctness),

Precision(How Accurate predictions are),

Recall (How many actual values are predicted),

F1 Score(balance between Precision and Recall.

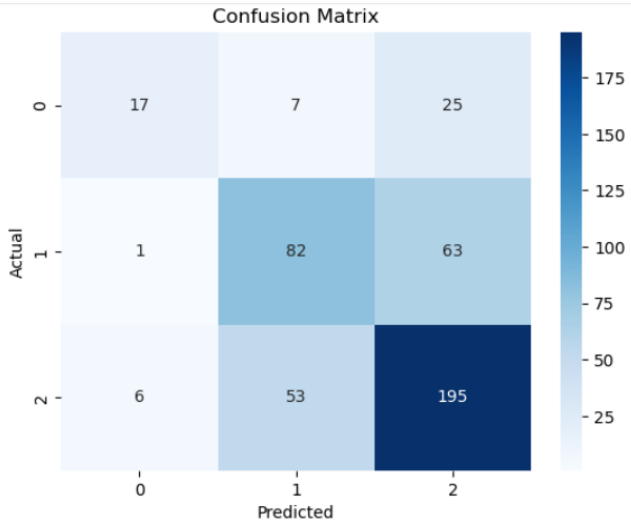
10. Then created confusion matrix for this:

```
cm = confusion_matrix(y_test, y_pred)

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix')
```

plt.show()

```
C:\Users\alpan\AppData\Local\Temp\ipykernel_14012\1816374961.py:5: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  df_model['skills'] = df_model['skills'].apply(lambda x: ' '.join(x))
C:\Users\alpan\AppData\Local\Temp\ipykernel_14012\1816374961.py:11: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  df_model['experience_level'] = le.fit_transform(df_model['experience_level'])
Accuracy: 0.6547884187082406
              precision    recall  f1-score   support

           0       0.71      0.35      0.47        49
           1       0.58      0.56      0.57       146
           2       0.69      0.77      0.73       254

    accuracy                           0.65       449
   macro avg       0.66      0.56      0.59       449
weighted avg       0.65      0.65      0.65       449
```

--> The model achieved an overall accuracy of 65% with strongest performance in predicting senior-level roles. However, lower recall for entry-level positions indicates class imbalance and less distinctive feature patterns for fresher roles



--> The model predicts Senior-level roles (class 2) with the highest accuracy, correctly classifying 150 instances.

--> Mid-level roles show reasonable prediction performance, but a significant number are misclassified as Senior roles.

--> The model struggles to accurately classify Entry-level roles, with many being misclassified as Senior roles.

--> The model tends to over-predict Senior-level roles, indicating a possible class imbalance or feature dominance.

The model demonstrates strong performance in identifying senior-level roles but shows limitations in distinguishing entry and mid-level positions, highlighting the need for improved feature engineering and class balancing.

11. Then saved the csv file into system

```python
import os
os.getcwd()
```

```
'C:\\Users\\alpan\\anaconda_projects\\d066b04f-01a6-47f1-bb97-d2502c45bac7'
```

```python
import pandas as pd

results = pd.DataFrame({
    'Class': ['Entry (0)', 'Mid (1)', 'Senior (2)'],
    'Precision': [0.71, 0.58, 0.69],
    'Recall': [0.35, 0.56, 0.77],
    'F1 Score': [0.47, 0.57, 0.73],
    'Support': [49, 146, 254]
})

results.to_csv("model_results.csv", index=False)
```

```python
accuracy = pd.DataFrame({'Metric': ['Accuracy'], 'Value': [0.67]})
accuracy.to_csv("accuracy.csv", index=False)
```

```python
from sklearn.metrics import confusion_matrix
import seaborn as sns

cm = confusion_matrix(y_test, y_pred)

cm_df = pd.DataFrame(cm,
                     columns=['Pred_0','Pred_1','Pred_2'],
                     index=['Actual_0','Actual_1','Actual_2'])

cm_df.to_csv("confusion_matrix.csv")
```

- **Created some more visualisations using Power BI**
- **Video Explanation:**
- **Video Link:**
  https://www.loom.com/share/72b82200293141ff8e91d4be0eff9556